

PROGRAM CONTROL STATEMENTS

Basics about Arrays

- ✓ **An array** is an indexed group of data storage locations that have the same name and are distinguished from each other by an index--**a number following the variable name, enclosed in brackets.**
- ✓ Like other C variables, arrays must be declared.
- ✓ An array declaration includes both the data type and the size of the array (the number of elements in the array).
- ✓ For example, the statement below declares an array named data that is of type int and has 1,000 elements:
- ✓ **int data[1000];**
- ✓ The individual elements are referred to by index as data[0] through data[999].
- ✓ The first element is data[0], not data[1].

- ✓ Each element of this array is equivalent to a normal integer variable and can be used the same way.
- ✓ The index of an array can be another C variable, as in this example:

```
int data[1000];
```

```
int count;
```

```
count = 100;
```

```
data[count] = 12;    /* The same as data[100] = 12 */
```

- ✓ **The above means that the element at index 100 of the data array is 12**

Controlling Program Execution

- ✓ The default order of execution in a C program is top-down.
- ✓ Execution starts at the beginning of the `main()` function and progresses, statement by statement, until the end of `main()` is reached.
- ✓ However, this order is rarely encountered in real C programs.
- ✓ The C language includes a variety of program control statements that let you control the order of program execution.
- ✓ You have already learned how to use C's fundamental decision operator, the `if` statement.

The for Statement

- ✓ The for statement is a programming construct that executes a block of one or more statements a certain number of times.
- ✓ It is sometimes called the **for loop** *because program execution typically loops through* the statement more than once.
- ✓ A for statement has the following structure:

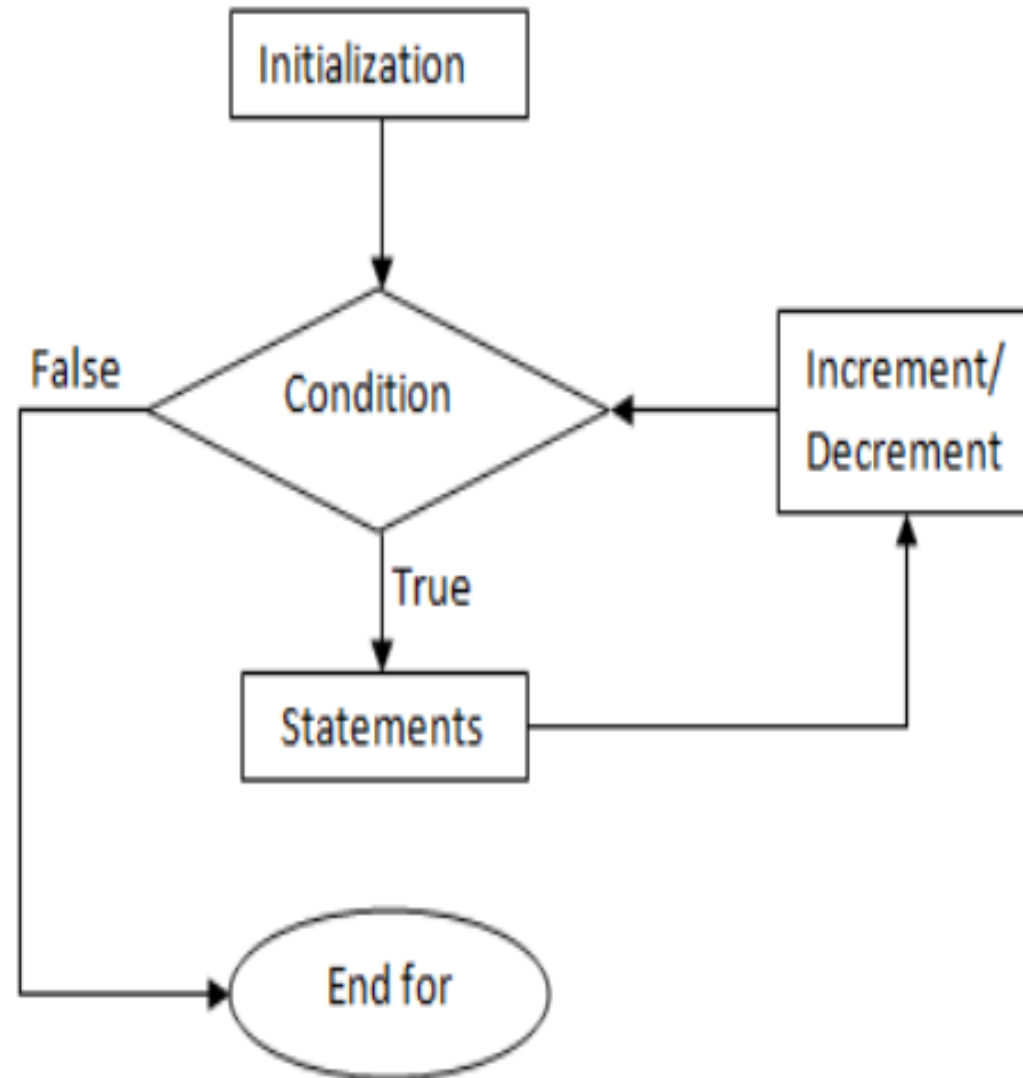
for (initial; condition; increment)
statement;

- ✓ *initial, condition, and increment are all C expressions, and statement is a single or compound C statement.*

✓ When a for statement is encountered during program execution, the following events occur:

1. The expression **initial is evaluated**. initial is usually an assignment statement that sets a variable to a particular value (starting point).
2. The **expression condition** is checked. condition is typically a relational expression.
3. If condition evaluates to fals, the for statement terminates, and execution passes to the first statement next statement (**outside the for loop**).
4. If condition evaluates to true , the C statement(s) in the for loop are executed.
5. The expression **increment is executed**, and execution returns to step 2.

Flow Chart of the for loop



Example program of the for loop

1		<code>#include <stdio.h></code>
2		<code>int main()</code>
3	☐	<code>{</code>
4		
5		<code>int x;</code>
6		
7		<code>for(x=0;x<=10;x++)</code>
8	☐	<code>{</code>
9		<code>printf("%d\n",x);</code>
10		<code>}</code>
11		
12		<code>printf("The Program has ended");</code>
13		
14		<code>return 0;</code>
15		<code>}</code>
16		

- ✓ The for statement is quite flexible.
- ✓ For example, you can omit the initialization expression if the test variable has been initialized previously in your program.
- ✓ **Note:** (You still must use the semicolon separator as shown)

```
count = 1;
```

```
for ( ; count < 1000; count++)
```

- ✓ The initialization expression doesn't need to be an actual initialization; it can be any valid C expression.
- ✓ Whatever it is, it is executed once when the for statement is first reached.
- ✓ For example, the following prints the statement **Now sorting the array...**

```
count = 1;
```

```
for (printf("Now sorting the array...") ; count < 1000; count++)
```

- ✓ You can also omit the increment expression and perform the updating in the body of the for statement.
- ✓ Again, the semicolon must be included. To print the numbers from 0 to 99, you could write

```
for (count = 0; count < 100; )
```

```
    printf("%d", count++);
```